

# EE272 Homework 7

## Floorplanning, Place & Route, and Signoff

Due: Feb 27, 2026, 11:59 pm

Released: Feb 19, 2026

In the previous assignments, you used Synopsys DC to synthesize the RTL description of your circuit into a gate-level netlist. Specifically, you compiled your design and mapped to standard cells in a specific technology. In this assignment, you will complete the flow and use commercial and open-source tools to take your synthesized netlist, place and route the gates, generate a layout, and verify it with a set of signoff steps. The output of the flow is a GDS file that contains the shapes (e.g., poly, metal) that represent your design and will (almost) be ready for tape-out.

First of all, make sure you are in the Singularity environment.

```
$ /farmshare/home/classes/ee/admin/software/bin/centos7.sh
```

To get started, first clone the homework repository:

```
$ git clone https://code.stanford.edu/ee272/dnn-accelerator-pnr.git
```

Then make sure you are at the top level of this code repo before running any commands:

```
$ cd dnn-accelerator-pnr
```

Next, set up the environment variables by sourcing the setup file. This setup is required for every new shell you open.

```
$ source setup.csh
```

The remainder of this handout will describe the GcdUnit pipe cleaner and the SramUnit pipe cleaner that you are already familiar with, but with both flows extended past synthesis, through each stage of place and route, and through signoff. We will also describe each stage of place and route that you will be running.

Your task is to take your final RTL for your DNN accelerator and push it through the full RTL-to-GDS flow while passing all signoff steps. By the end, you will hopefully have a

GDS that can be fabricated and that meets an area, energy, and timing target. This is a big accomplishment!

# 1 Pipe Cleaners – GcdUnit and SramUnit

Pipe cleaners are small designs that run through the flow quickly and help to identify errors early. It is good practice to frequently run pipe cleaners while developing the flow before running your full design. We will be reusing the same two pipe cleaners from the previous assignments for GcdUnit and SramUnit. These two flows now include steps for each stage of place and route and for each signoff step.

In this section, we will provide a high-level overview of running the entire pipe cleaner before doing deeper dives into the Innovus Foundation Flow, how mflowgen nodes for Innovus are set up, how to run each step, and which scripts and reports are the most important to inspect.

## 1.1 High-level Overview

To start the GcdUnit pipe cleaner, first create a build directory and configure for this design:

```
$ cd GcdUnit
$ mkdir build
$ cd build
$ mflowgen run --design ../design
$ make status
```

Then open the mflowgen graph to see what this pipe cleaner flow looks like:

```
$ make graph
(open graph.pdf in a PDF viewer like evince)
$ evince graph.pdf
```

Here is the list of steps:

```

$ make status
- build -> 0 : constraints
- build -> 1 : info
- build -> 2 : rtl
- build -> 3 : skywater-130nm
- build -> 4 : testbench
- build -> 5 : rtl-sim
- build -> 6 : gen-saif-rtl
- build -> 7 : synopsys-dc-synthesis
- build -> 8 : cadence-innovus-flowsetup
- build -> 9 : cadence-innovus-init
- build -> 10 : cadence-innovus-power
- build -> 11 : cadence-innovus-place
- build -> 12 : cadence-innovus-cts
- build -> 13 : cadence-innovus-postcts_hold
- build -> 14 : cadence-innovus-route
- build -> 15 : cadence-innovus-postroute
- build -> 16 : cadence-innovus-signoff
- build -> 17 : mentor-calibre-gdsmerge
- build -> 18 : open-magic-def2spice
- build -> 19 : synopsys-pt-timing-signoff
- build -> 20 : synopsys-ptpx-rtl
- build -> 21 : gl-sim
- build -> 22 : netgen-lvs-def
- build -> 23 : open-magic-antenna
- build -> 24 : open-magic-drc
- build -> 25 : open-magic-gds2spice
- build -> 26 : gen-saif-gl
- build -> 27 : mentor-calibre-comparison
- build -> 28 : netgen-lvs-gds
- build -> 29 : synopsys-ptpx-gl

```

Feel free to cross-check the `construct.py`, the graph visualization, and the step configuration files (<https://github.com/mflowgen/mflowgen/tree/master/steps> to see which files are passing between which steps.

Here is a high-level overview of the Innovus steps. The “cadence-innovus-flowsetup” node first uses a script generator to generate the base scripts used in each of the downstream Innovus nodes (also see the section on the Cadence Innovus Foundation Flow later in this handout). The “cadence-innovus-init” step starts place and route by reading in the design. Each of the subsequent nodes reads in a checkpoint (i.e., an Innovus database), does something (e.g., runs placement), and outputs another checkpoint. At the end of the flow, the “cadence-innovus-signoff” step dumps all outputs of place and route. These outputs include the final gate-level netlist, GDS, and other files, which are used to run further signoff steps (e.g., DRC and LVS). Notice that a GDS merge step is needed to merge the GDS from place

and route (which includes only the wires) with the GDS of the standard cell library.

You can run the GcdUnit pipe cleaner through to signoff like this:

```
$ make status # check which step signoff is
$ make 16     # assuming signoff is step 16
$ make status # signoff should be marked done
```

Each of the Innovus steps can be brought up on a GUI with a debug target like this:

```
$ make debug-16 # brings up Innovus GUI after signoff
```

You should also get a sense of the runtime of each step:

```
$ make runtimes
```

You can run the SramUnit pipe cleaner in the same way:

```
$ cd SramUnit/build
$ mflowgen run --design ../design
$ make status # check which step signoff is
$ make 16     # assuming signoff is step 16
```

Take the time to bring up the GUI after each of the steps and compare them to learn what each step does.

## 1.2 The Innovus Foundation Flow

Relevant mflowgen step: “cadence-innovus-flowsetup”.

Cadence Innovus comes with its own flow generator called the Innovus Foundation Flow. The mflowgen steps that we provide for Innovus use the foundation flow to generate a base set of scripts that execute each of the major steps in place and route (e.g., init, place, cts, route, postroute, signoff). Each script is connected to a downstream mflowgen step that wraps the script and enhances it for convenience and debuggability.

There is a major benefit to relying on the Cadence Innovus Foundation Flow for the canonical commands. As Cadence updates its version of Innovus year after year, their recommended options to achieve more optimal quality of results change. Some options are deprecated, other options are newly suggested, and new commands are introduced. Using the foundation flow generator allows us to easily leverage the most up-to-date, Cadence-recommended commands with no maintenance costs of our own.

You can dump the foundation flow yourself by running the “writeFlowTemplate” command in an Innovus shell. This is essentially what the “cadence-innovus-flowsetup” node

is responsible for (see the step configuration in <https://github.com/mflowgen/mflowgen/blob/master/steps/cadence-innovus-flowsetup/run.sh>). Feel free to open Innovus and run it yourself:

```
$ innovus
>>> writeFlowTemplate
```

The generator will dump files into the current directory. The master script is “SCRIPTS/gen\_flow.tcl”. The “cadence-innovus-flowsetup” mflowgen node runs this master script and generates the base scripts for the following steps:

- Design initialization – innovus-foundation-flow/INNOVUS/run\_init.tcl
- Placement – innovus-foundation-flow/INNOVUS/run\_place.tcl
- Clock tree synthesis (CTS) – innovus-foundation-flow/INNOVUS/run\_cts.tcl
- Post-CTS hold-fixing – innovus-foundation-flow/INNOVUS/run\_postcts\_hold.tcl
- Route – innovus-foundation-flow/INNOVUS/run\_route.tcl
- Postroute – innovus-foundation-flow/INNOVUS/run\_postroute.tcl
- Signoff – innovus-foundation-flow/INNOVUS/run\_signoff.tcl

The output of this node is the entire “innovus-foundation-flow” directory, which feeds into all downstream Innovus nodes. Feel free to open each of these scripts to see how the canonical place and route commands are run. Pay particular attention to the super commands (e.g., placeDesign, optDesign).

### 1.3 mflowgen – Sub-Modularization Within Innovus Steps

The mflowgen steps we provide for Innovus are designed to be submodular. This means that each step (which is already modular on its own) is further split into individual scripts for various purposes (e.g., reporting, setting up variables and margins, tweaking constraints). An mflowgen parameter called “order” then allows the user to parameterize how the step is run from a Python interface. For example, the user can remove a script that is not needed for their design, add an additional custom script for their design, or simply define a reordering of the existing scripts.

Take a look at the design initialization step (i.e., “cadence-innovus-init”), which is responsible for reading in the post-synthesis design files from Synopsys DC and executing floor-planning. This step’s configuration file (<https://github.com/mflowgen/mflowgen/blob/master/steps/cadence-innovus-init/configure.yml>) lists the following inputs and outputs:

- (input) innovus-foundation-flow – Scripts generated by the Innovus Foundation Flow.
- (input) adk – ASIC design kit interface to the process technology and library files.
- (input) design.v – The post-synthesis gate-level netlist.
- (input) design.sdc – Constraints dumped from synthesis.
- (output) design.checkpoint – The working Innovus database after the step finishes.

The configuration file also defines the “order” parameter, which lists scripts in the following order (note that the contents of these scripts can be found in the “scripts” subdirectory):

- main.tcl – Main script from the Innovus Foundation Flow for design initialization.
- quality-of-life.tcl – Useful variables and settings when working with Innovus.
- floorplan.tcl – Creates the floorplan with the core area and all margins.
- add-endcaps-welltaps.tcl – Add well taps if ADK contains well taps
- pin-assignments.tcl – Places IO pins along the perimeter of the design.
- make-path-groups.tcl – Creates timing path groups to help the timing engine prioritize certain paths.
- reporting.tcl – Dumps a variety of reports.

The order parameter determines the order in which the scripts are run. For example, floorplanning (“floorplan.tcl”) will be executed after we read in the design (“main.tcl”) and set up quality-of-life variables (“quality-of-life.tcl”), with reporting done at the end of the step (“reporting.tcl”). The order parameter can be accessed as a normal python list in your mflowgen graph defined in construct.py. For example, the parameter can be printed like this:

```
>>> init = Step( 'cadence-innovus-init', default=True )
>>> order = init.get_param( 'order' )
>>> print( order )
['main.tcl', 'quality-of-life.tcl', 'floorplan.tcl',
'pin-assignments.tcl', 'make-path-groups.tcl', 'reporting.tcl']
```

The commands run Innovus with a special script that simply loops through the order parameter and runs each item one by one. It searches for each script in both the step’s local “scripts” directory (for the default versions of the scripts) as well as the “inputs” directory (where the user can supply custom versions of the scripts) with priority given to the inputs. This allows users to extend the step with new scripts.

For example, if we had a node called “custom-init” that provided “new-last-step.tcl” as an output, we could parameterize the init step to append the new script at the end:

- main.tcl
- quality-of-life.tcl
- floorplan.tcl
- pin-assignments.tcl
- make-path-groups.tcl
- reporting.tcl
- **new-last-step.tcl** (a new custom script)

The mflowgen construct script would use the mflowgen API for extending the list of inputs in a step (i.e., `extend_inputs()`) and extend the init step inputs with the list of all outputs of our custom node (i.e., `all_outputs()`). Running “make graph” would then produce a graph with a new input edge into the init step like this:

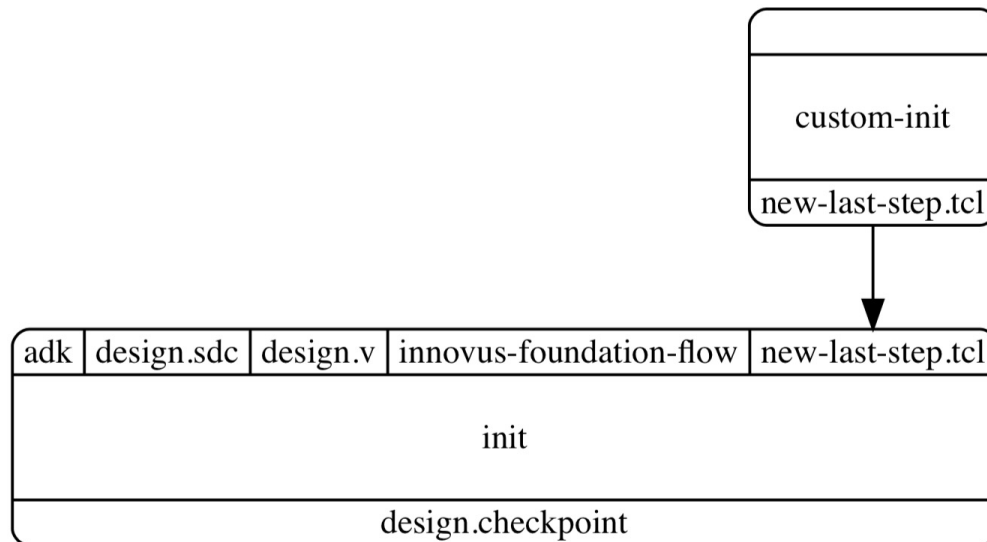


Figure 1: Adding custom nodes.

The mflowgen construct script that does the above would look like this:

```
init      = Step( 'cadence-innovus-init', default=True )
custom_init = Step( ... ) # Comes from somewhere and
                        # has an output 'new-last-step.tcl'

init.extend_inputs( custom_init.all_outputs() )

g.add_step( init )
g.add_step( custom_init )
g.connect_by_name( custom_init, init )
```

Finally, we would parameterize the order of the init step to include the new input script:

```
order = init.get_param( 'order' )
order.append( 'new-last-step.tcl' ) # Append the script to run last
init.update_params( { 'order': order } )

print( order ) # -> [ 'main.tcl', ..., 'new-last-step.tcl' ]
```

Note that we could simply have replaced the entire init node with our own custom version. However, it can be very useful to reuse the majority of an existing step while tweaking just a small part of it.

In this assignment, you will be tweaking the **init** step (to add a custom floorplan and pin assignments) and the **power** step (to customize your power strategy for your macros).

## 1.4 Design Initialization and Floorplanning

The first step during place and route is called “init” (i.e., the “cadence-innovus-init” step) and reads in the design from synthesis and executes floorplanning. Refer back to 1.3 to see what the inputs, outputs, and scripts are and what they do. Here is a list of scripts you might tweak at this step:

- floorplan.tcl – This is where we size the core area (targeting a certain density) and aspect ratio. This is also where we place macros. A reasonable density target is about 75%, and a very aggressive density is over 90%. Aspect ratios that are square are generally preferred over long rectangular ones. Rectangular floorplans place more stress on either horizontal or vertical routing resources and also make it difficult to place macros. Note that “planDesign” is the command that automatically places your macros.
- pin-assignments.tcl – This is where we use `editPin` to spread pins along the sides on specific metal layers. We could also read a pre-saved io file in this script.
- make-path-groups.tcl – You can create path groups to tell the timing engine to prioritize certain paths that you may find more important than others (e.g., paths to macros). During timing optimization, the tool loops through each of the path groups and tries to fix the worst paths in each group. If there were only a single path group, the tool might never work on paths further down the list that you as a designer know are important. Enough useful path groups have been set for you that you will likely not need to add any more.

**Note:** You should create or reuse the `mflowgen` nodes available in the `SramUnit` that customize “floorplan.tcl” and “pin-assignments.tcl”.

Here is a list of checks you will want to run through before moving on to the next step:



- “reports/preplace.summary.gz” – The timing summary report at the end of init assumes no wire delays. This means that the timing reports should look nearly identical to those from synthesis. Therefore, the first thing you should check is that the timing is not be significantly better or worse than in synthesis. When wire delays are factored in, these numbers will only look worse, so the tool should not be trying too hard to meet timing at this point. In particular, there should **never** be negative slack in this report. (Note: you can use `gunzip preplace.summary.gz` to decompress the file)

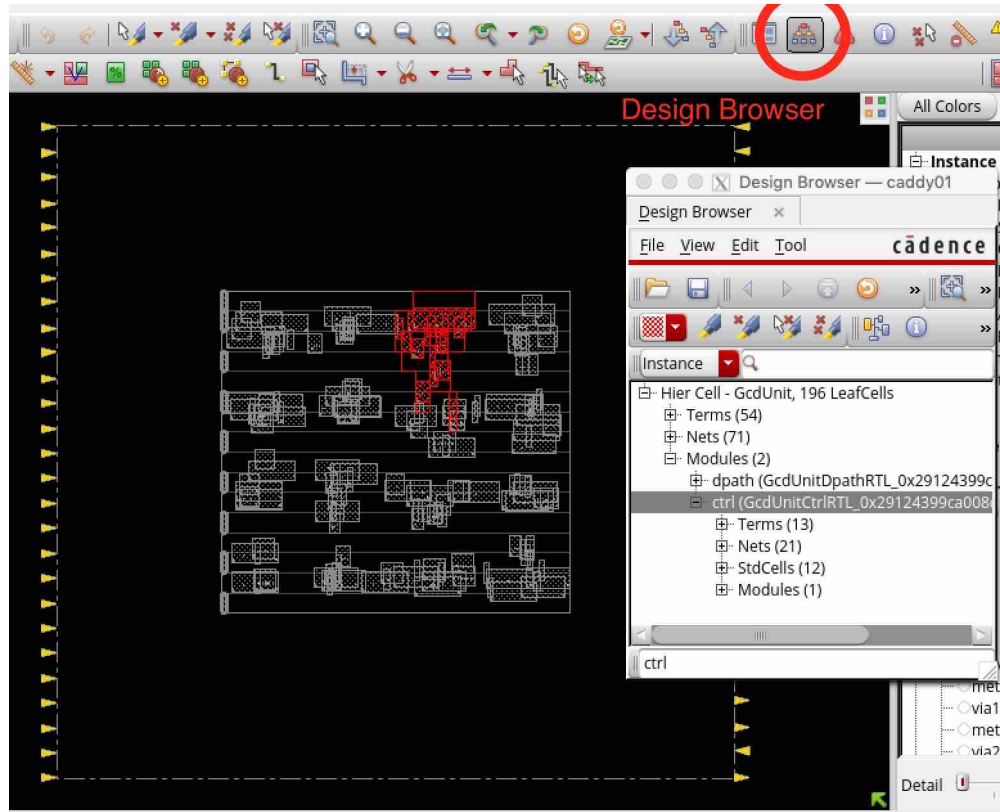
| timeDesign Summary |       |         |           |         |  |
|--------------------|-------|---------|-----------|---------|--|
| Setup mode         | all   | reg2reg | reg2cgate | default |  |
| WNS (ns):          | 1.740 | 1.740   | 3.160     | 2.209   |  |
| TNS (ns):          | 0.000 | 0.000   | 0.000     | 0.000   |  |
| Violating Paths:   | 0     | 0       | 0         | 0       |  |
| All Paths:         | 54    | 34      | 2         | 52      |  |
| analysis_default   | 1.740 | 1.740   | 3.160     | 2.209   |  |
|                    | 0.000 | 0.000   | 0.000     | 0.000   |  |
|                    | 0     | 0       | 0         | 0       |  |
|                    | 54    | 34      | 2         | 52      |  |

Density: 70.339%

- “logs/run.log” – There should be no errors in the logs. Most possible errors at this stage will be related to reading the technology files and libraries.
- Check the floorplan in the GUI – Run the debug target on this step and bring up the GUI. GcdUnit does not have any macros, but this is generally the time to make sure that the macros are placed at reasonable locations and orientations (e.g., pins facing the core area, no narrow channels). It can be useful to do a fast prototype of cell placement to see what the design will look like with stdcells. A floorplan mode placement does this by placing cells while ignoring DRCs for speed:

```
> setPlaceMode -place_design_floorplan_mode true
> place_design
```

- You can also open the “Design Hierarchy” viewer and click on a module to see where the stdcells in a particular module were placed. For example, the control module for GcdUnit was randomly placed in a clump near the top in this floorplan-mode prototype placement.



## 1.5 Power Strategy

The next step is for power strategy (i.e., the “cadence-innovus-power” step) and sets up the power rings, stripes, and grid. Here is a list of what the default inputs, outputs, and scripts are and what they do:

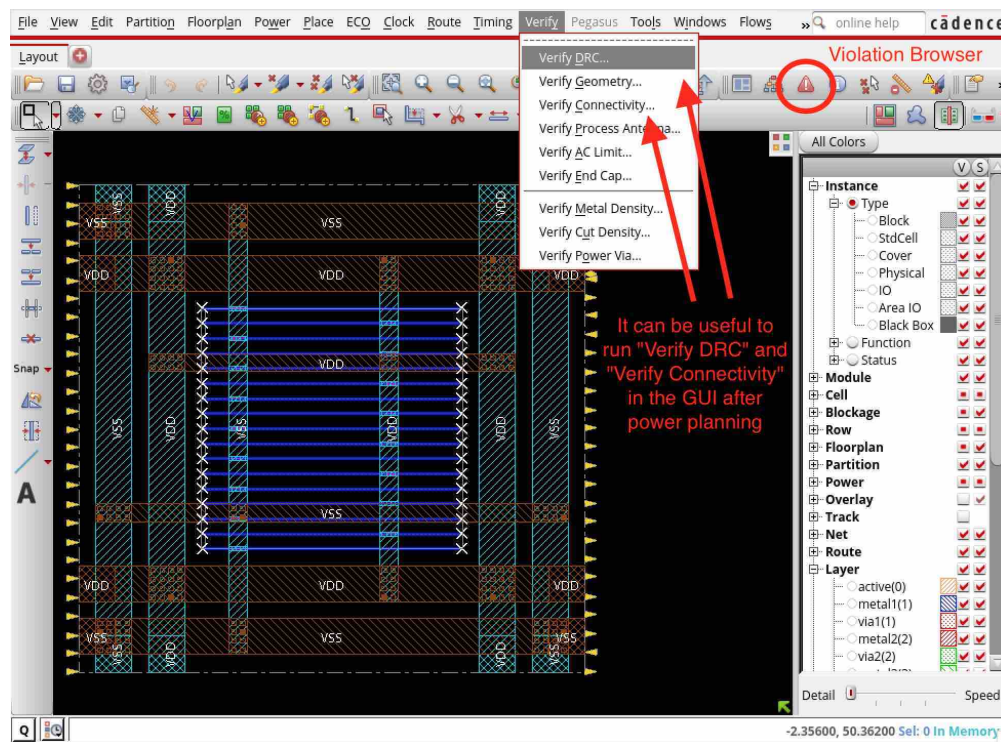
- (input) design.checkpoint – The working Innovus database from the previous step.
- (output) design.checkpoint – The working Innovus database after the step finishes.
- globalnetconnect.tcl – Logically connects power nets (e.g., VDD and VSS) to pins on cells in the design, even if their pins are not explicitly called VDD or VSS.
- main.tcl – Main script that calls a power strategy.
- power-strategy-dualmesh.tcl – Builds a fine mesh on lower metals and a coarse mesh on higher metals (you will not use this script in 130nm).
- power-strategy-singlemesh.tcl – Builds a single power mesh on higher metals.

You should create or reuse the mflowgen nodes available in the SramUnit that customize “globalnetconnect.tcl” and “power-strategy-singlemesh.tcl” in your design.

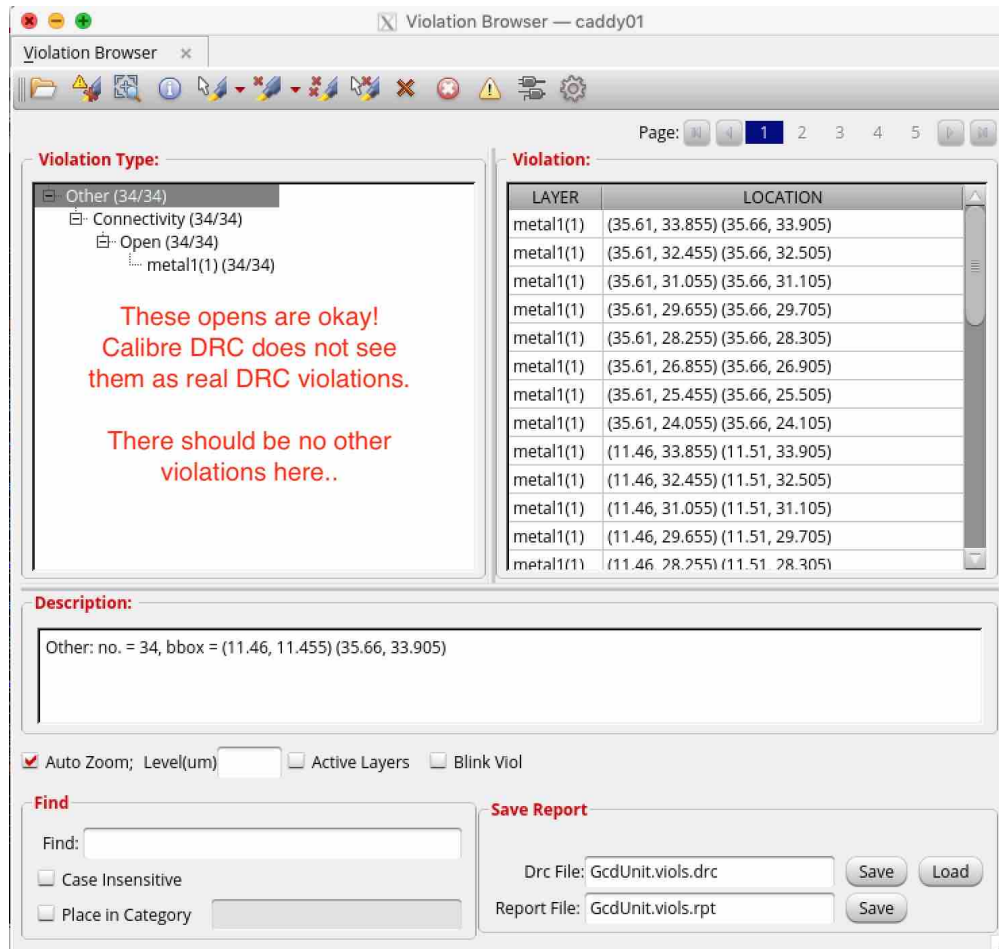
To iterate on your power strategy, we highly recommend opening the GUI (running the debug target for init gives you a clean slate) and copy-pasting the power commands one by one while tweaking them. You should press “U” to undo if you do not like the result of an addStripe or addRing command. You can also just type the “undo” command. If you want to start over, you can run the following command to delete power-related shapes:

```
> deleteAllPowerPreroutes
```

At any point in your work, you can run an Innovus DRC which only checks the shapes that Innovus can see (i.e., wires). After opening “Verify DRC” just accept the default settings. You can also run an Innovus LVS with “Verify Connectivity”, although this will always fail at this stage because the cells and wires do not exist yet which certainly does not match the source netlist.



The violation browser is the red triangle button and lists all violations found by these checks. This view should be clear of DRC violations.



## 1.6 Placement

The next step is placement (i.e., the “cadence-innovus-place” step), which iterates on different placements with the global router. From the place step and onwards, you should not expect to need to change any scripts. Please post on Ed Discussion if you realize a need to do so (e.g., to fix final hold violations).

Here are the inputs, outputs, and scripts in the place step and what they do.

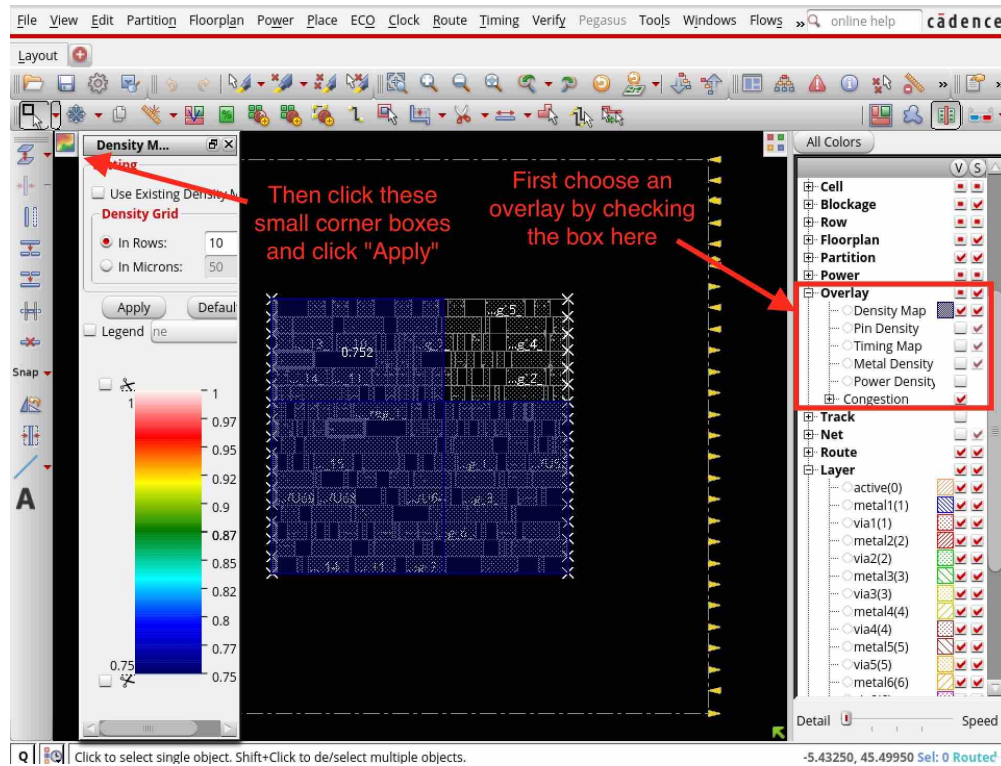
- (design.checkpoint same as before)
- setup-cellpad.tcl – Sets up padding next to each DFF to reserve space for buffers during timing optimization. Without padding in very dense designs, there may not be enough space left for timing-fixing buffers.
- main.tcl – Calls the Innovus Foundation Flow script.
- clean-cellpad.tcl – Removes the padding constraint now that placement is over.

Here is a list of checks you will want to run through before moving on to the next step:

- reports/place.summary – With wires in place, the timing will be worse than it was in init and synthesis. The timing must look good in this report. The goal of all future steps is only to *preserve* the timing in this report.
- logs/run.log – Look for the final congestion analysis table in the log. Make sure that the overflows are at most a few percent. If the routing tracks are too oversubscribed (across all GCells in the design), the resulting congestion will make timing very hard to meet. The table below is nearly clean because there is no congestion in GcdUnit.

| Congestion Analysis By layer: (blocked Gcells are excluded)       |     |           |          |
|-------------------------------------------------------------------|-----|-----------|----------|
|                                                                   |     | OverCon   |          |
|                                                                   |     | #Gcell    | %Gcell   |
| Layer                                                             |     | (1)       | OverCon  |
| -----                                                             |     |           |          |
| metal1                                                            | (1) | 0( 0.00%) | ( 0.00%) |
| metal2                                                            | (2) | 1( 0.09%) | ( 0.09%) |
| metal3                                                            | (3) | 0( 0.00%) | ( 0.00%) |
| metal4                                                            | (4) | 0( 0.00%) | ( 0.00%) |
| metal5                                                            | (5) | 0( 0.00%) | ( 0.00%) |
| metal6                                                            | (6) | 0( 0.00%) | ( 0.00%) |
| metal7                                                            | (7) | 0( 0.00%) | ( 0.00%) |
| -----                                                             |     |           |          |
| Total                                                             |     | 1( 0.02%) | ( 0.02%) |
| Overflow after earlyGlobalRoute (GR compatible) 0.00% H + 0.00% V |     |           |          |
| Overflow after earlyGlobalRoute 0.00% H + 0.00% V                 |     |           |          |

- Check density and congestion overlays in the GUI – Open the debug target for place. Then enable overlays as shown in the following figure. This is a visual version of what you can already find in the logs.



## 1.7 Clock Tree Synthesis

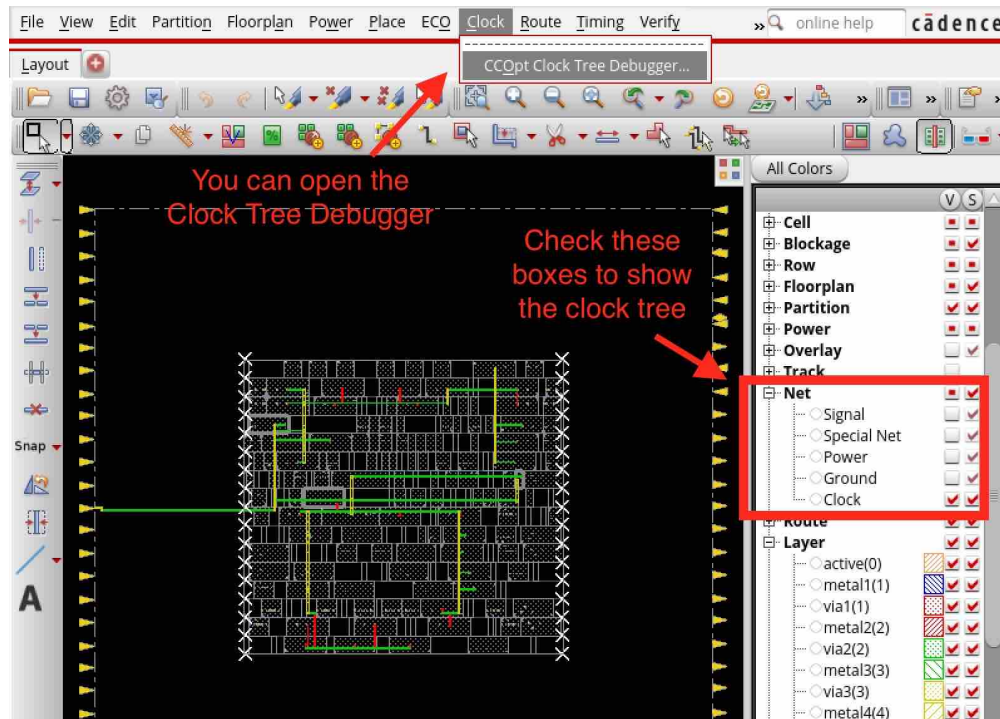
The next step is clock tree synthesis (i.e., the “cadence-innovus-cts” step), which skew-balances the clock tree. Here are the inputs, outputs, and scripts and what they do. Again, you should not need to change any of these.

- (design.checkpoint same as before)
- setup-ccopt.tcl – These are options for clock tree optimization that you will probably not need to touch.
- main.tcl – Calls the Innovus Foundation Flow script.

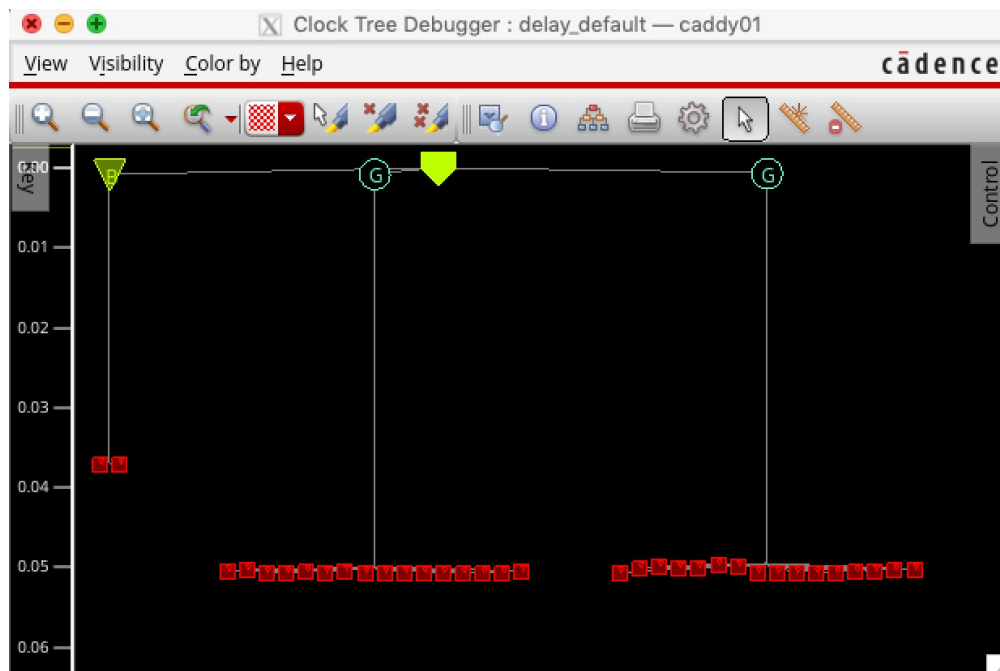
Here is a list of checks you will want to run through before moving on to the next step:

- Look at the clock tree in the GUI – You can check boxes in the right panel of the GUI to show just the clock tree. This can give you a good visual sense of how the clock is being distributed. Notice how the clock enters from the middle pin of the left side in the GcdUnit layout. If your clock pin was placed in an odd corner during floorplanning, it could affect your clock tree.





- Look at the clock tree debugger in the GUI – Open the “Clock Tree Debugger” to see a tree of all clock gates, buffers, and sinks in your design. The y-scale is the clock insertion delay for that cell (i.e., how long it takes for the clock to propagate from the pin to that cell).



## 1.8 Route, Postroute, and Signoff

The final steps are less involved from a designer perspective. These steps run detailed route (i.e., the “cadence-innovus-route” step) and loop until timing is met or until the tool quits (i.e., the “cadence-innovus-postroute” step). Then the final timing analysis is run and output files are written out (i.e., the “cadence-innovus-signoff” step). Here are the inputs, outputs, and scripts and what they do. You should not expect to change these.

- (design.checkpoint same as before)
- (output) design.gds.gz – The GDS layout with all wires and empty holes for the stdcells. The GDS merge step that comes next will combine this one with the stdcell.gds in the ADK (and any SRAM gds) to create the final GDS.
- (output) design.lvs.v, design.vcs.v, and design.virtuoso.v – These are slightly different versions of the post-place-and-route gate-level netlist for different purposes. The LVS version removes physical-only cells (e.g., fillers) that would cause spurious LVS failures. The Virtuoso version removes decaps that significantly slow down SPICE simulation.
- (output) design.lef – The LEF view of your final design. This can be used to insert your design into a larger design in another flow.
- (output) design.pt.sdc – The constraints that Innovus used, meant to tell the timing signoff engine in Synopsys PT what constraints to use.
- (output) design.spf.gz – This file has parasitic RC values for every net in your design. This is used in timing signoff and power estimation.
- (postroute) setup-optmode.tcl – This is where you might set up slack targets for setup/hold. If you are having trouble with small amounts of negative slack, you can set numbers here.
- (signoff) generate-results.tcl – This script writes out all the design files after place and route is over.

Here is a list of checks you will want to run through:

- reports/signoff.area.rpt – Compare this area report to the report from synthesis. This area will be higher because it includes not just the gates but also margins, empty space (filler), halos, buffers, and any overheads from floorplan restrictions.
- logs/run.log – Search for “violations” and make sure the violation count is zero. This is an automated Innovus DRC that is equivalent to clicking “Verify DRC” in the GUI. Also check the connectivity report in the log right next to the DRC. This is Innovus running LVS on the wires and making sure connections in the layout match those in the netlist.



- reports/signoff.summary – The numbers in this report should be similar to those in the place summary report. The timing engine that generated these numbers is signoff quality, so the exact numbers will be slightly different.
- reports/signoff\_hold.summary – This is the hold timing report. This report must only have positive numbers!
- Pull up the GUI – This is a good chance to just look at your final layout and check if anything odd catches your eye. Feel free to do any of the checks from previous steps again.

## 2 Post-Place-and-Route Signoff

After Innovus has completed place and route, the design passes to a set of signoff tools which independently verify the design.

### 2.1 Design Rule Check (DRC)

Design rule check (DRC) is an important step to make sure that your layout follows the rules provided the foundry. If your design doesn't pass this check, it is highly possible that your design will not be manufactured correctly. To run DRC check, we use an open-source tool called **Magic**. Here are the two inputs to this step:

- (input) design\_merged.gds – A merged GDS containing the Innovus GDS, the stdcell GDS, and any macro GDS.
- (input) adk – This step uses the DRC rule deck from the ADK.

If your design contains DRC errors after running the open-magic-drc node, you will see something like:

```

Loading DRC CIF style.
Cell YOUR_DESIGN_NAME has 1134880 error tiles.
Total DRC errors found: 0
Cell YOUR_DESIGN_NAME has 1134880 error tiles.
Total DRC errors found: 0

```

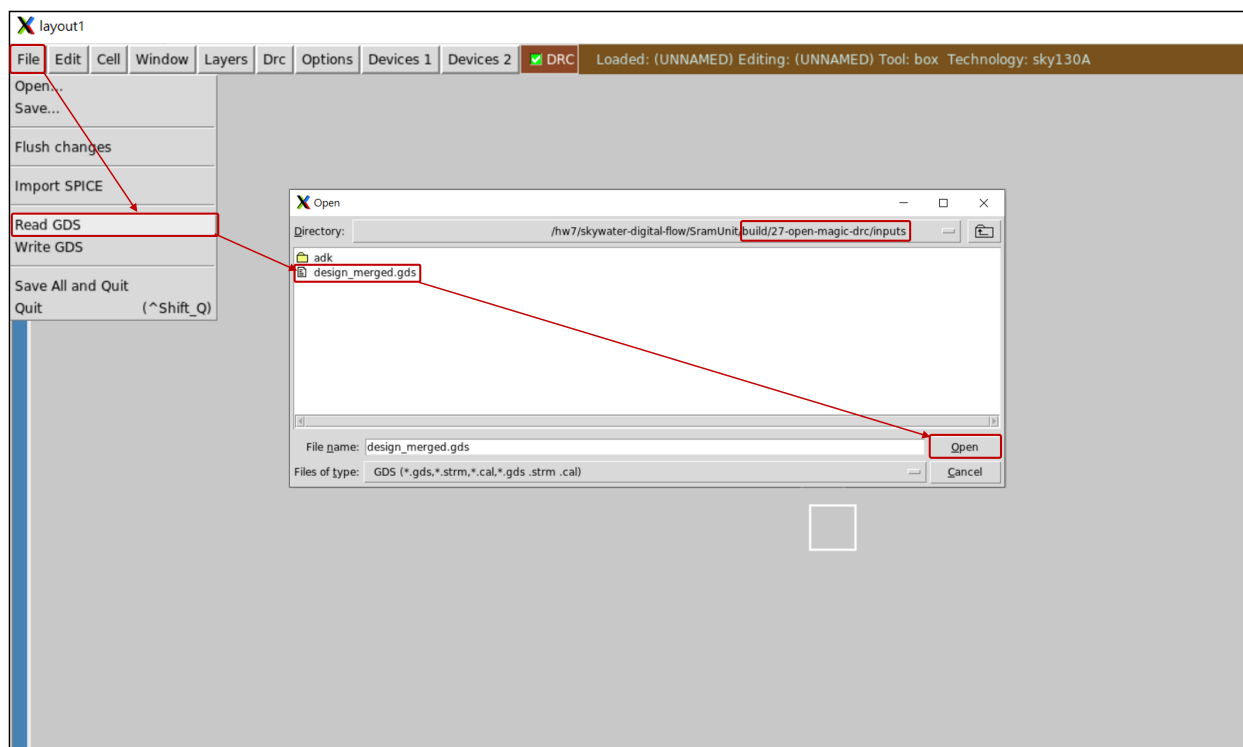
Note that you cannot trust this message: **Total DRC errors found: 0**. For some reason Magic still prints these messages even if you have DRC errors in your design. Your design is only considered DRC free if you don't see any error tiles in your report:

```
Loading DRC CIF style.  
Total DRC errors found: 0  
Total DRC errors found: 0
```

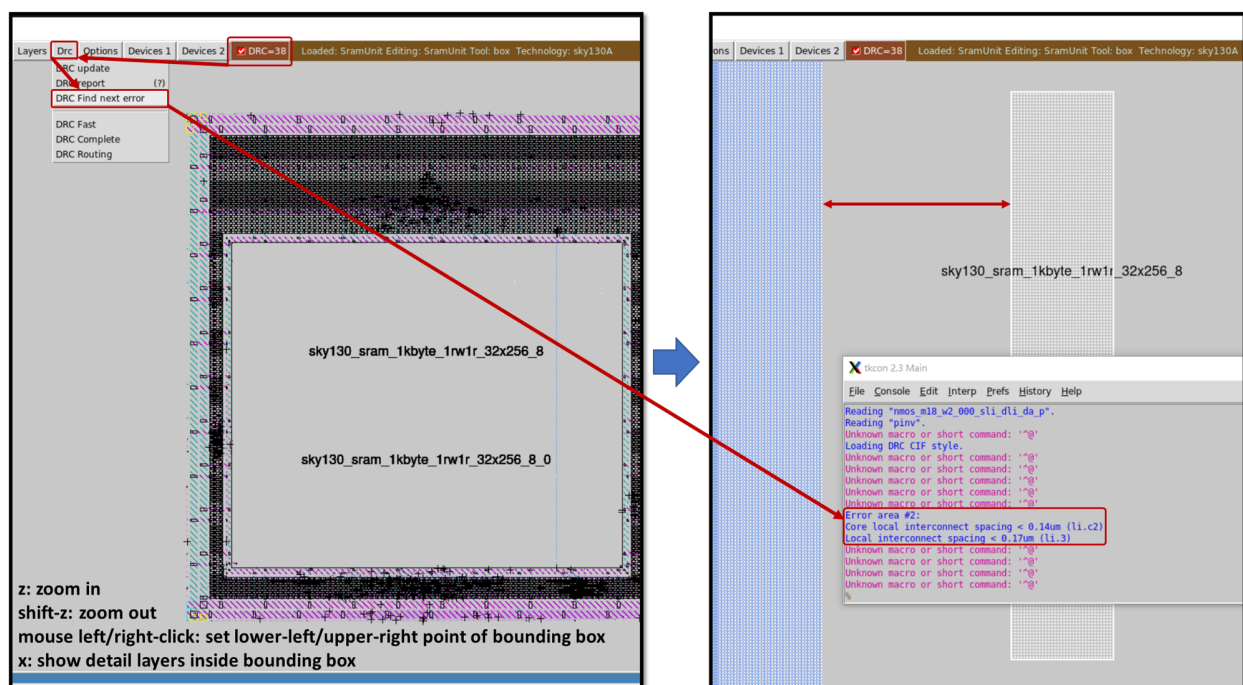
If you see some error tiles, you will need to open the magic GUI and locate these errors. Suppose your open-magic-drc node is 27-open-magic-drc, you can bring the GUI up by:

```
$ cd build/27-open-magic-drc  
$ magic &
```

After the GUI pops up, read the GDS file from your input folder:



It will take some time for the tool to analyze your GDS. If your design contains DRC errors, the green box at top will turn red and report the number of violations in your design.



You can navigate through errors by clicking DRC->DRC find next error. Magic will take you to the DRC error and print error message on its console. You can move around the windows with arrow keys and zoom-in/zoom-out with z and shift-z. If you want to inspect the details, you can set the bounding box to cover the region of interest and press x to reveal detail layers. You should be able to find out where the DRC errors are and why they exist.

You will need to fix the DRC errors if they are created by innovus. However, it is possible to find some DRC errors that are inside a SRAM macro. Since SRAM macros have their own special rules that magic does not know, so it is okay to ignore them. In fact, we already did this by reading its LEF file to treat it like a black box.

## 2.2 Layout-vs-Schematic (LVS)

Next we run LVS also with Mentor Calibre (i.e., the “mentor-calibre-lvs” step). Here are the inputs, outputs, and scripts and what they do.

- (input) rules.svrf – An optional file with additional LVS commands to run.
- (input) design\_merged.gds – This must be the same merged GDS that you ran DRC on. The combination of DRC and LVS tells you that the layout is manufacturable (DRC) and that that same layout matches the schematic (LVS) that you run gate-level tests on. If you run DRC and LVS on different layouts, then this semantic link is broken.
- (input) design.lvs.v – The post-place-and-route gate-level netlist with physical-only cells removed.

- (input) adk – This step uses the Calibre LVS rule deck.

Open the LVS report to check the comparison result:

- lvs.report – This will have a smiley face at the top if you pass LVS. The most important table to look at in the lvs.report is the “Information and Warnings” table part of the way down. This will show the number of ports, nets, and devices found in the layout and in the source and how the numbers match up.

```
*****
                                     INFORMATION AND WARNINGS
*****
```

|            | Matched<br>Layout | Matched<br>Source | Unmatched<br>Layout | Unmatched<br>Source | Component<br>Type |
|------------|-------------------|-------------------|---------------------|---------------------|-------------------|
| Ports:     | 56                | 56                | 0                   | 0                   |                   |
| Nets:      | 566               | 566               | 0                   | 0                   |                   |
| Instances: | 15                | 15                | 0                   | 0                   | MN(NMOS_VTL)      |
|            | 16                | 16                | 0                   | 0                   | MP(PMOS_VTL)      |
|            | 20                | 20                | 0                   | 0                   | SPDW_2_1          |
|            | 3                 | 3                 | 0                   | 0                   | SPDW_2_1_1        |
|            | 4                 | 4                 | 0                   | 0                   | SPDW_2_2          |
|            | 1                 | 1                 | 0                   | 0                   | SPDW_2_2_1        |
|            | 14                | 14                | 0                   | 0                   | SPDW_3_1          |
|            | 23                | 23                | 0                   | 0                   | SPUP_2_1          |
|            | 1                 | 1                 | 0                   | 0                   | SPUP_2_1_1        |
|            | 1                 | 1                 | 0                   | 0                   | SPUP_2_2          |
|            | 1                 | 1                 | 0                   | 0                   | SPUP_2_2_1        |
|            | 18                | 18                | 0                   | 0                   | SPUP_2_2_2        |
|            | 14                | 14                | 0                   | 0                   | SPUP_3_1          |
|            | 329               | 329               | 0                   | 0                   | _invv             |
|            | 2                 | 2                 | 0                   | 0                   | _invx2v           |
|            | 12                | 12                | 0                   | 0                   | _nand2v           |
|            | 1                 | 1                 | 0                   | 0                   | _nand3v           |
|            | 1                 | 1                 | 0                   | 0                   | _nand4v           |

If you see a mismatch, open the debug target for this step to bring up the GDS viewer GUI (Calibre DESIGNrev). After the layout loads, press > many times to increase the depth and see inside all the cells.

Unfortunately, LVS mismatches are very difficult to debug because of how LVS transforms and squashes transistors together, causing a mismatch in one place to manifest in another. The debug GUI will still be useful to list which nets and ports Calibre thinks are missing.

### 3 Documentation/Help

These tools are complicated and documentation is a tremendous help. For this assignment, the most useful reference will likely be the Innovus user guide [/cad/cadence/INNOVUS20.13.000.lnx86/doc/innovusUG/innovusUG.pdf](#) and command reference [/cad/cadence/INNOVUS20.13.000.lnx86/doc/innovusTCR/innovusTCR.pdf](#).

Another good way to get help is on the Innovus command line. If you see a command in one of the provided scripts and you're not sure what it means, enter `man *name_of_command*`. This will typically give you a basic description of what the command does and what required or optional arguments it accepts. You can do the same thing for more information on error/warning messages, substituting the error code for the command name.

### 4 Your Task

Your job is to modify the existing steps for your project. Some (but not necessarily all) of the changes you'll need to make are as follows:

- We have provided a 4×4 DNN accelerator that you will use for your design in the rtl node. This is to reduce physical design runtimes and match the fact we are using much smaller memories.
- In this assignment you will have 12 SRAM macros. 4 4KB macros for the input and weight buffers (2 each) and 8 1KB macros for the accumulation buffer. These are much smaller than we would like (compared to sizes from your previous assignments), but are the memory macros we have generated for now. These are the macros you will floorplan.
- Typically 0.70 is a good density target (**floorplan.tcl**) for chip design. However, in this homework you only need to achieve a density target to 0.55 for full points.
- You will receive 80/100 points for reaching signoff with a timing clean design at 20ns and 0.30 density on your 4×4 accelerator. To receive 100/100 points, you should reach a timing clean design at 10ns and 0.55 density at signoff.
- Please make sure you don't see any DRC or Connectivity violations at the signoff step. However, if the error is in our known issues section (for example, open vss nets on power rails or open vss connections on pwell) then it is okay.
- Although we talked about some post-signoff checks (i.e. open-magic-drc and mentor-calibre-lvs) in the handout, we do not ask you do complete them in this assignment because there are still some issues haven't been addressed in the Skywater technology. However, you can still try to run those steps and get a taste of those post-signoff checks.

- **Tip:** If you want to learn more about any command, type **man “command”** in the Innovus command shell. This will bring up the manual for that command and should help you understand what exactly it is doing.

A few words of advice: when you’re modifying the flow, make sure your modification worked before you try to run an entire place and route! Iteration times can be very long, so you don’t want to waste time waiting for a run to finish, when you may have introduced a bug in the first part of the flow. It can be useful to enter commands one-by-one and see how they affect your design in the GUI.

## 5 Known Issues

Since the both our physical design flow and the SkyWater130 open source PDK are very new, there are some known issues because of bugs in the PDK and in the flow. If you run into these for either the pipecleaners or your own design, do not panic, you won’t be penalized for them, and you will still produce a relatively clean design. If you figure out how to solve some of these issues, please do let us know and we will incorporate them into the repository.

### 1. **Design:** GcdUnit, SramUnit

**Step:** cadence-innovus-signoff when it runs verifyConnectivity

**Issue:** Net VSS: has an unconnected terminal, has special routes with opens, has regular routing with opens.

293 Problem(s) (IMPVFC-96): Terminal(s) are not connected.

1 Problem(s) (IMPVFC-200): Special Wires: Pieces of the net are not connected together.

41 Problem(s) (IMPVFC-92): Pieces of the net are not connected together.

**Cause:** In the stdcells LEF files VPB and VNB pins are on pwell and nwell layers. However, unlike nwell, pwell doesn’t extend through the entire width of the cell. Innovus is not able to detect whether the connection has been established. But in gds-based DRC check, it can be seen that the pwell is shorted to VGND through mcon.

### 2. **Design:** GcdUnit, SramUnit

**Step:** open-magic-drc (You are not required to run this step but we have provided it for your future reference.)

**Issue:** Several errors of the type “Error while reading cell “sky130\_fd\_sc\_hd\_\_clkinv\_4” (byte position 222126): Unknown layer/datatype in boundary, layer=236 type=0”.

**Cause:** This does not seem to be a critical issue, as DRC still runs fine. These errors are happening when magic is reading in standard cell GDSs. Likely a bug in SkyWater PDK.

### 3. **Design:** SramUnit

**Steps:** First appears in cadence-innovus-place and some remain through cadence-innovus-signoff.

**Issue:** Design rule violations on transition times (max\_tran in DRV table reported in the logs and signoff summary report). Place log says:

**\*info:** Total 24 net(s) have violations which can't be fixed by DRV optimization. MultiBuffering failure reasons

**\*info:** 24 net(s): Could not be fixed because the gain is not enough.

**Cause:** If we look at signoff.tran report, it shows us the ports on which these transition time violations are happening, and they all happen to be the addr or wmask pins on the SRAM, and it looks like the cell driving them does not meet the 0.04 rise transition time and there is about -0.006 negative slack on all these pins. We must find out where this transition time constraint is set and whether it is reasonable. This does not happen in GcdUnit.

## 6 Resolved Issues

### 1. **Design:** GcdUnit, SramUnit

**Steps:** gl-sim, gen-saif-gl, synopsys-ptpx-gl (You are not required to run these steps but we have provided them for your future reference.)

**Issue:** We have not yet figured out how to cleanly run gate level simulation (GLS), and are currently working on it. Once GLS runs, gen-saif-gl and synopsys-ptpx-gl should work without issues.

**Solution:** To get the gate level netlist to compile, we had to fix several bugs in the SkyWater PDK:

(1) Change ``default_nettype none` to ``default_nettype wire` in several Verilog primitive definitions.

(2) The cell `dlxbn/sky130_fd_sc_hd_dlxbn.behavioral.pp.v` has a wire that did not do anything that was named with a number. We removed it.

After these changes the simulation ran, but we were only seeing x's. It turns out that in order to get the models to work correctly we had to use the standard cell Verilog with power and ground, and use the functional instead of the behavioral models. This entailed a few changes in the testbench that we have now pushed to the repo. The behavioral models and netlists without power and ground in the SkyWater PDK still have bugs.

### 2. **Design:** SramUnit

**Steps:** netgen-lvs-gds

**Issue:** v2lvs errors out.

**Cause:** There were two errors. The first one was because some wires in the gate-level netlist had the same names if interpreted in a case-insensitive manner, for example "n10" and "N10" were different wires. The problem happens because SPICE is case-insensitive. These wires were being introduced in DC, so we had to change `compile.tcl` in the DC node to use `-case_insensitive` with `define_name_rules` command before dumping the netlist.

The second error happened because the SPICE netlist for the SRAM does not have

buses at the interface but the Verilog instantiation does, so v2lvs complains. Flattening the buses when writing out Verilog for LVS in Innovus signoff fixed this issue (`generate_results.tcl, saveNetlist -flattenBus`).

3. **Design:** GcdUnit, SramUnit

**Step:** cadence-innovus-postroute

**Issue:** ERROR: (IMPCCOPT-1209): Non-leaf slew time target of 0.133ns is too low on the rising edge. The largest clock gate is unable to drive the largest inverter in power\_domain auto-default. To adhere to the given slope target, you will need to select a stronger clock gate, increase the slew target to at least 0.133ns or remove these driver cells from the CTS cell lists: `sky130_fd_sc_hd__clkinv_16`.

**Cause:** We are trying to find out if we are setting an unreasonable slew target somewhere inadvertently. This error can be solved by not allowing Innovus to use this large inverter (`setDontUse`), but to resolve it cleanly, we must understand why Innovus is using a large inverter in the first place.

4. **Design:** GcdUnit, SramUnit

**Step:** cadence-innovus-signoff

**Issue:** ERROR: (IMPLF-109): Cannot create OBS on overlap layer for rectilinear partition 'GcdUnit' because there is no overlap layer defined in any LEF file.

**Cause:** Likely a bug in SkyWater PDK technology LEF. **Solution:** Solved by adding these line to the `rtk-tech.lef` after the last metal layer:

```
LAYER OVERLAP
TYPE OVERLAP ;
END OVERLAP
```

## 7 Code Submission and Grading

Please submit the following in a `.tar.gz` file to Canvas:

- Mflowgen place and route scripts.
- Your final GDS.
- Area, timing, and power reports. The timing report must not report any timing violations (all slacks must be positive).
- DRC (`verifyDrc`) and LVS (`verifyConnectivity`) reports from the cadence-innovus-signoff step. DRC should be clean. Connectivity report should not report any issues other than the known issues.

Please write the following in a PDF report and submit it on Canvas:



- Your strategy/approach. Your report should describe and justify the modifications you made to the existing pipe cleaner place and route flow to make it work for the DNN accelerator.
- A summary of your design's area, frequency and power. You should compare these to your post-synthesis results.

Name the .tar.gz and PDF files using your and your partner's full names. The grading of this assignment will be based whether or not you were able to to successfully place and route your design and the quality of your report.